

Part I. Random Forest Optimization

(tuned) Random Forest Model - All Stations, 2010s

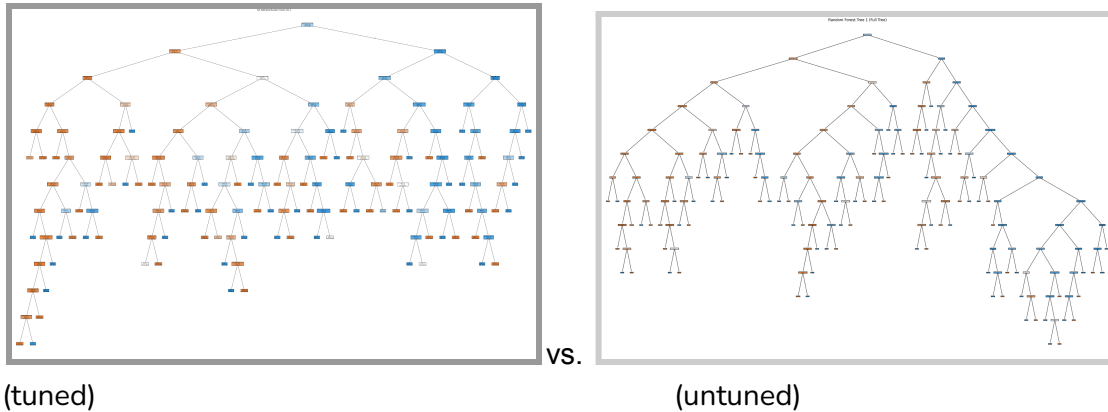
Hyperparameter Tuning & Model Comparison

Key Differences			
What Changed?	Untuned Value	Tuned Value	Effect
Criterion	'gini'	'entropy'	Entropy often gives slightly better splits when classes are imbalanced or nuanced
max_depth	None	14	Prevents trees from growing too deep (overfitting)
min_samples_split	2	4	Slightly more conservative in splitting nodes — improves generalization
n_estimators	100	108	Marginal increase in ensemble strength
Others	Same	Same	max_features and min_samples_leaf stayed unchanged

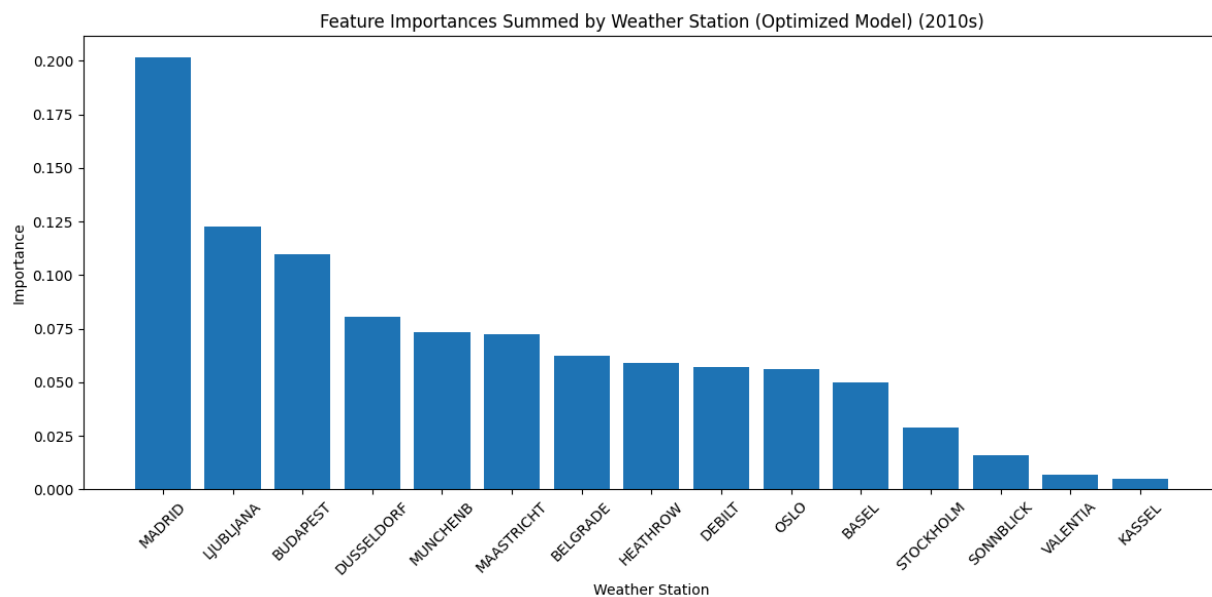
To enhance model performance and prevent overfitting, we employed Randomized Search to optimize hyperparameters of the Random Forest Classifier. Unlike exhaustive grid search, randomized search efficiently samples a broad range of parameter values, enabling quicker convergence on high-performing configurations. Our tuning revealed that the best model used the entropy criterion for splitting; this criterion can better capture information gain in complex or imbalanced datasets, compared to the Gini impurity index (which is prone to focusing on the majority class early on, and overlooking the minority). The optimal max_depth was set to 14, which allowed trees to grow deep enough to learn meaningful patterns without overfitting. Increasing min_samples_split from 2 to 4 prevented the model from splitting nodes with too few samples, promoting more stable and generalizable trees. The number of estimators slightly increased, from 100 to 108, striking a balance between performance and computational cost; the few additional trees are probably what account for the fractional increase in accuracy. Other parameters, like max_features and min_samples_leaf, stayed at their defaults, indicating lesser impact in this context.

When comparing the untuned and tuned models, the tuned model exhibited more nuanced tree structure with subtle additional branches, suggesting a capacity to capture more complex relationships.

Amy Zhang
2.4 Hyperparameter Tuning Report



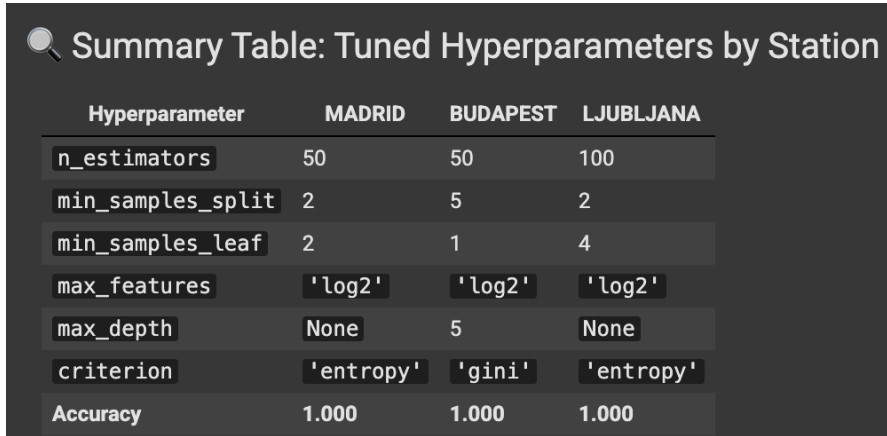
Feature rankings remained consistent overall, although the untuned model prioritized Budapest over Ljubljana, whereas the tuned model reversed their order, giving Ljubljana a more pronounced edge.



This may reflect the similarity in “weather fingerprint” between the stations, with the tuned model’s additional trees uncovering critical differentiating patterns absent from the untuned baseline. Though the accuracy gains were slight, we can trust this more to perform more reliably than its baseline when generalizing to new data.

(tuned) Random Forest Models - Barcelona, Ljubljana, Budapest (all years)

Individual Station-Level Random Forests



A summary table titled "Summary Table: Tuned Hyperparameters by Station" showing the optimal hyperparameters for Random Forest models across three stations: Madrid, Budapest, and Ljubljana. The table lists seven hyperparameters: n_estimators, min_samples_split, min_samples_leaf, max_features, max_depth, criterion, and Accuracy. The values for each hyperparameter are consistent across all three stations, with the exception of min_samples_split and min_samples_leaf which vary slightly. The Accuracy is 1.000 for all stations.

Hyperparameter	MADRID	BUDAPEST	LJUBLJANA
n_estimators	50	50	100
min_samples_split	2	5	2
min_samples_leaf	2	1	4
max_features	'log2'	'log2'	'log2'
max_depth	None	5	None
criterion	'entropy'	'gini'	'entropy'
Accuracy	1.000	1.000	1.000

For the top three most influential weather stations — **Madrid**, **Ljubljana**, and **Budapest** — we applied **Randomized Search** to optimize the hyperparameters of each Random Forest model individually. While the untuned model (with 100 estimators and scikit-learn's default settings) had already achieved 100% accuracy across these stations, hyperparameter tuning remains a valuable pre-validation step. It can surface configurations that are **more computationally efficient** (e.g., setting a finite `max_depth` rather than leaving it as `None`) and potentially **more robust to overfitting** (e.g., tuning `min_samples_split` to avoid shallow, noisy splits).

The randomized search revealed distinct model profiles for each weather station, confirming that one-size-fits-all parameters may overlook local dynamics. For instance, **Madrid** and **Budapest** required only 50 trees for optimal performance, whereas **Ljubljana** benefited from a larger ensemble of 100 — potentially indicating more complex feature interactions in its weather patterns.

There were notable differences in **splitting behavior**:

- `min_samples_split` (the minimum number of samples needed to split a node):
 - Madrid: 2
 - Ljubljana: 2
 - Budapest: 5 → A more conservative splitting threshold, likely helping to regularize and simplify the tree structure for a more uniform dataset.
- `min_samples_leaf` (the minimum number of samples required to be at a leaf node):

- Madrid: 2
 - Budapest: 1
 - Ljubljana: 4 → Suggests that Ljubljana's trees needed broader, more stable terminal nodes to prevent overfitting on spiky patterns.
 - **max_depth:**
 - Madrid & Ljubljana: **None** → Fully grown trees, indicating useful deep patterns.
 - Budapest: 5 → A shallow model sufficed, again hinting at more homogeneous weather–pleasantness relationships.
 - **criterion:**
 - Madrid & Ljubljana: **Entropy**
 - Budapest: **Gini**
This suggests that for Budapest, a greedy class-purity approach was sufficient, while for the other two, the finer granularity of **information gain** helped discriminate between more subtly different conditions.
 - **max_features:**
All three used **'log2'**, which restricts the number of features considered at each split — a regularization mechanism that often improves generalization.
-

Feature Importance & Tree Structure

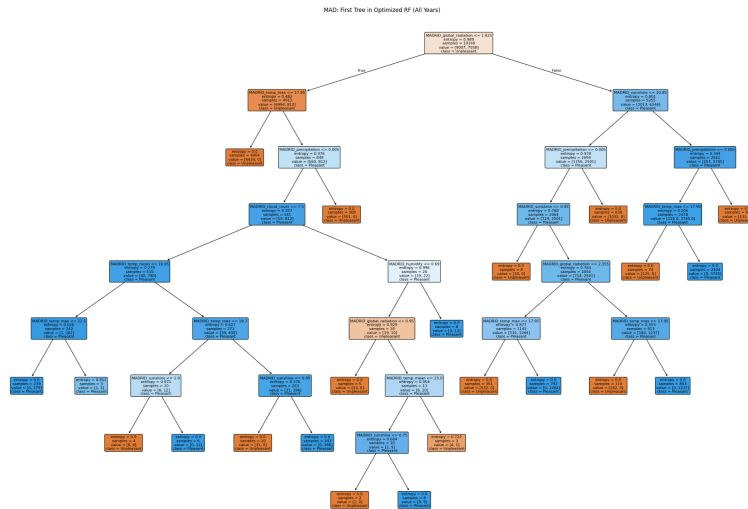
Despite some reshuffling, the **top 3 most influential weather features** remained largely consistent across stations. All three models ranked **precipitation**, **temp_max**, and **temp_mean** among their top contributors, albeit in varying orders. Notably, **Ljubljana's model no longer ranked global radiation** among its top features — aligning its profile more closely with Budapest's, both in feature influence and decision logic.

Visual inspection of the decision trees reinforces these insights:

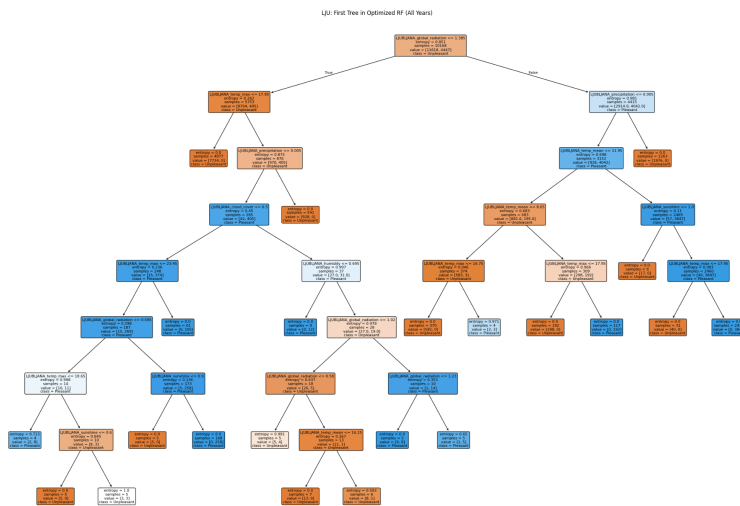
- **Madrid** and **Ljubljana** show complex, deeply nested branches — reflecting the nuanced interactions among multiple weather variables.
- **Budapest's tree** is significantly shallower and less branched, confirming that a simpler decision boundary suffices for accurate classification.

Amy Zhang

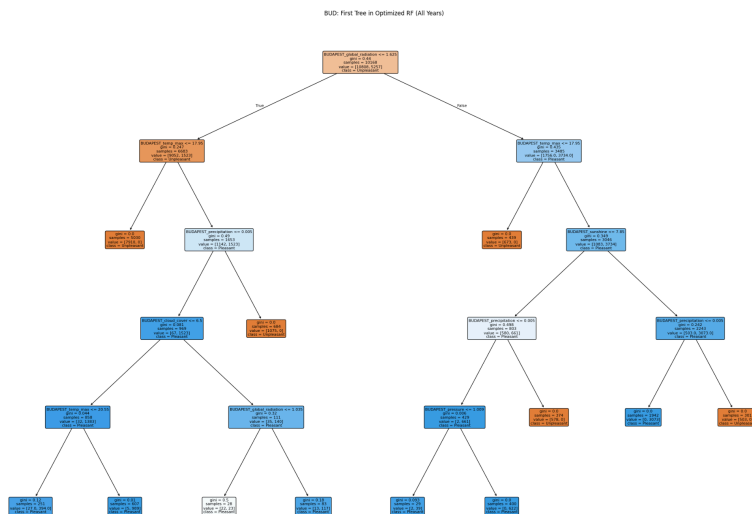
2.4 Hyperparameter Tuning Report



(Madrid)



(Ljubljana)



(Budapest)

Final Thoughts

Hyperparameter tuning in this context yielded models that are not only accurate but **tailored** — both in complexity and behavior — to the unique structure of each station's weather data. The improved regularization, split constraints, and individualized model depth may not change accuracy on seen data, but they strengthen the model's reliability and interpretability for generalization to new time periods or unseen locations.

Part II. Deep Learning Optimization

Untuned Model Recap:

To predict the "pleasantness" classification for any of the 15 weather stations, we structured our dataset to reflect temporal dependencies: each prediction was based on the prior 7 days of weather data. This made Recurrent Neural Networks (RNNs), and specifically Long Short-Term Memory (LSTM) models, an appropriate choice due to their ability to capture temporal sequences and propagate learned context over time.

The untuned LSTM model included:

- **Memory units:** 64
- **Dropout:** 0.3 (for regularization)
- **Dense layer:** 32 units with ReLU activation (to introduce non-linearity and mitigate vanishing gradients)
- **Output layer:** 1 unit with sigmoid activation (for binary classification)
- **Loss function:** Binary Cross-Entropy (suitable for binary classification)
- **Optimizer:** Adam (adaptive gradient-based optimizer that combines momentum and RMSprop)

The model was trained for 20 epochs with a batch size of 32. At first glance, the model performed well, achieving:

- **Test accuracy:** ~95%
- **Final loss:** ~0.12

However, the binary class imbalance (many more "unpleasant" than "pleasant" samples) made accuracy a misleading metric. Deeper evaluation via a classification report and confusion matrix revealed:

- **F1-score (majority class):** 0.97
- **F1-score (minority class):** 0.61
- **False Positive Rate (minority):** ~43%
- **AUC (ROC):** 0.97

These findings indicated that the model excelled at predicting the majority class but struggled with the minority. We experimented with threshold tuning (adjusting the decision boundary of the sigmoid output), which improved the minority class F1-score to 0.65—but this post-training adjustment did not impact the model’s actual learning process.

Tuned Model (Bayesian Optimization):

We then applied **Bayesian Optimization** to perform hyperparameter tuning. Unlike random or grid search, Bayesian tuning **uses information from previous trials** (i.e., probabilistic models of performance) to intelligently explore the hyperparameter space and reduce search redundancy.

Tuning Parameters:

- **Memory units:** 32 to 256 (step of 32)
- **Dropout:** 0.0 to 0.5 (step of 0.1)
- **Learning rates:** [1e-4, 1e-3, 1e-2]
- **Epochs per trial:** 10
- **Total trials:** 15

Note: The LSTM model used `return_sequences=False`, meaning it outputs a single vector rather than one per timestep—sufficient for our binary classification task. Despite having only **one LSTM layer**, the model still qualifies as “deep learning” due to its layered representation learning and non-linear transformations.

Best Parameters Found:

```
Best hyperparameters:  
units: 96  
dropout: 0.4  
learning_rate: 0.001
```

While it might seem contradictory to increase both memory units and dropout, it's important to understand that model performance isn't just a tug-of-war between underfitting and overfitting. Increased memory units don't necessarily mean the model is just memorizing more — rather, they provide the capacity to better capture complex temporal patterns in the data. In this case, more memory may have allowed the model to express patterns that were previously out of reach.

At the same time, that increase in expressive power can raise the risk of overfitting, particularly if the model begins to fit too tightly to the noise in the training set. The higher dropout rate helps counterbalance this, enforcing regularization and encouraging the model to generalize under more variable or incomplete input conditions.

👉 Sound Design Analogy:

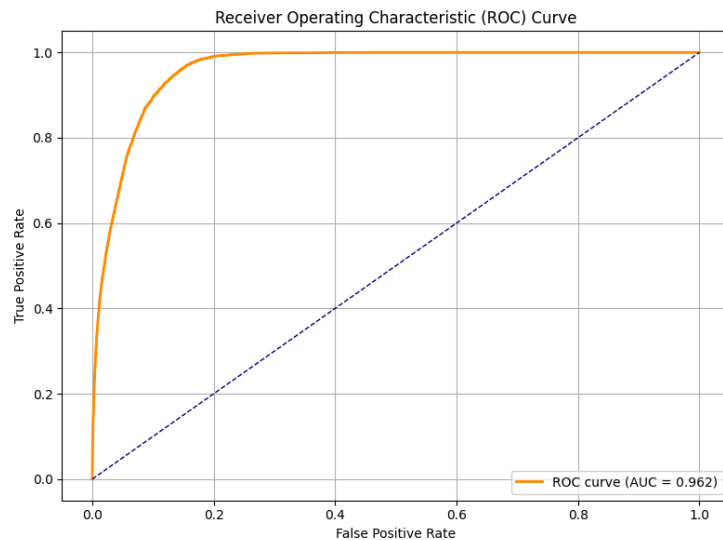
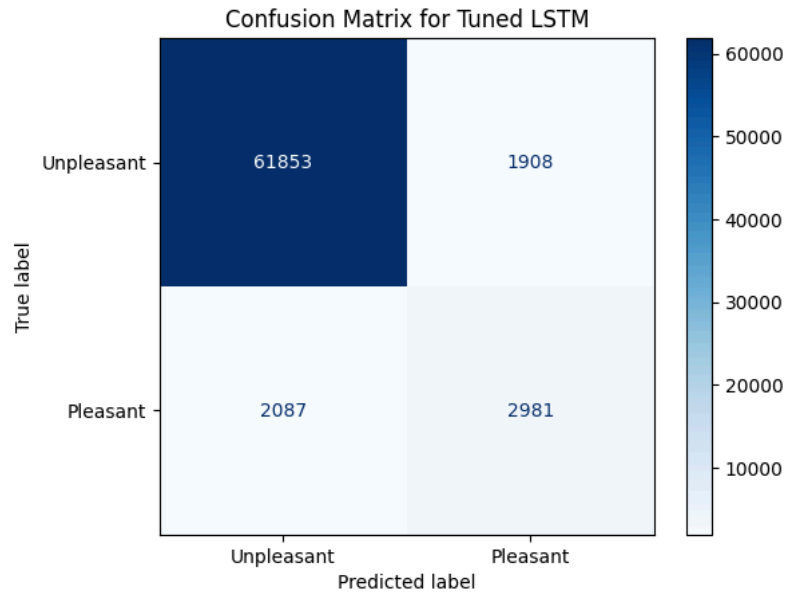
It's like mixing audio using **EQ** and **compression**. EQ lets you emphasize or suppress different frequencies (like memory units letting the model attend to different signal patterns), while compression controls dynamic range — keeping the volume from spiking or dropping too far (like dropout preventing overamplification of specific weights or noise). Boosting frequencies (adding more neurons) can make things sound rich but also messy; you might **increase compression** (dropout) to keep the signal clean. Both tools shape the final sound — not by canceling each other out, but by **balancing expression and restraint**.

Tuned Model Performance:

Despite the optimized hyperparameters, the tuned model performed **slightly worse**:

Classification Report:				
	precision	recall	f1-score	support
Unpleasant	0.97	0.97	0.97	63761
Pleasant	0.61	0.59	0.60	5068
accuracy			0.94	68829
macro avg	0.79	0.78	0.78	68829
weighted avg	0.94	0.94	0.94	68829

- **Test accuracy:** 0.942 (vs. 0.946)
- **F1-score (minority class):** 0.60 (vs. 0.61)
- **AUC:** 0.962 (vs. 0.97)



Hypothesis:

A plausible explanation is that the tuned model converged to a **local minimum** in the loss surface. Because Bayesian Optimization balances exploration and exploitation, it's possible that it narrowed in too quickly on a suboptimal region. Additionally, the limited trial count (15) and shallow network depth may have limited the optimizer's ability to escape poor configurations.

These results underscore the importance of establishing a robust baseline before engaging in tuning, as optimization alone cannot compensate for architectural or data-related limitations.

Part 3: Iteration

A focused iteration on the **Valentia weather station** could provide valuable insights for addressing class imbalance challenges, as this station has the lowest support for the minority class and consistently shows the weakest F1 score performance. By isolating data from this station, we can tailor strategies that are highly specific to its unique “weather fingerprint” and operational patterns.

Model choice:

Starting with a **Random Forest** model here is advantageous because it offers interpretable insights into feature importance and decision splits. Running a **Randomized Search** hyperparameter tuning could help determine whether **Gini impurity** or **Entropy** better suits splitting criteria for this specific station’s data characteristics.

Feature focus and data simplification:

To improve model focus and reduce noise, narrowing down to the three most consistently influential weather variables — **precipitation**, **temp_max**, and **temp_mean** — could make the model both more robust and interpretable. More aggressive tuning of **class weights** or even applying techniques like **SMOTE** (Synthetic Minority Over-sampling Technique) on this smaller feature set could mitigate imbalance issues more effectively (* ‘SMOTE’ is a ChatGPT suggestion; to be researched).

Iteration on Deep Learning:

Following initial Random Forest insights, a deep learning model (such as a tuned LSTM) can be applied on this refined dataset to explore temporal dependencies in these key variables. This could highlight subtle sequence patterns in weather changes that influence flight safety, which classical models might miss.

Variables of Critical Importance for Air Ambulance Flight Decisions

From both the Random Forest’s feature importance analyses and domain intuition, the following variables stand out as key factors for assessing flight safety: **Precipitation**, **Maximum Temperature (temp_max)**, **Mean Temperature (temp_mean)**. Focusing on these variables will likely give Air Ambulance teams actionable, reliable indicators when deciding whether it’s safe to fly, while also providing a manageable input space for iterative model tuning.